rather than a point-to-point network.

In a user-space VPN, the IP packets from a TUN/TAP adapter are encrypted and then encapsulated into UDP and sent over the Internet. At the destination, the remote host de-encapsulates, decrypts and authenticates these IP packets before pushing them into a TUN/TAP virtual adapter.

Configuring OpenVPN

There are various ways to configure OpenVPN and you can choose the one that matches your requirement. In this article we will look at a few types of configurations that are typically used.

To start with, install OpenVPN as follows. The server set-up in this article uses a Linux system, while various clients can be used for testing purposes, like Windows XP with openvpn-gui; Fedora and Ubuntu with an OpenVPN plug-in for NetworkManager; as well as Debian, CentOS and Red Hat with command line-based OpenVPN clients.

We will start with installing the software from the distributions repository.

On CentOS 5.3 I had activated the Fedora EPEL repository [*https://fedoraproject.org/wiki/EPEL*] for the OpenVPN package to make sure that I used a 2.1 version build.

```
# yum install openvpn
```

The configuration files are kept in the */etc/openvpn* directory. The *init* script installed with the package takes care of registering the OpenVPN with the *chkconfig* system, so that it can be started and stopped using the *service* command.

While starting the openvpn service, the *init* script takes care of loading all the configuration files from the */etc/openvpn* directory with the *.conf* extension. If a particular configuration requires some initialisation to be done, then we can create a script in */etc/openvpn/* by the same name as the configuration file with a *.sh* extension. So, for example, let's call our configuration file */etc/openvpn/server.conf*. We can create a script */etc/openvpn/server.sh* that will be executed before loading the configuration from the *server.conf* file.

We will use the following directory structure for our set-up, so let us create it:

```
# mkdir /etc/openvpn/keys
```

Regardless of the configuration chosen, there are a few options used in all types of configurations. Although some of these are default settings, it is a good idea to specify them in the configuration file. The following example is the static key configuration, which is the most basic type, so I will describe everything about it thoroughly; from thereon, for other configurations, just the additions and removals will be specified.

Initially we only allow the client and server to ping each other. After reviewing all popular configurations we will have a look at how to provide the clients access to the private network behind the VPN, and also how to configure a proxy to let the clients surf the Net securely.

Note: On the CentOS Server SELinux was enabled, so the server set-up described here should work on the SELinux-enabled boxes. At certain places I have shown the output displaying the SELinux contexts.

Static key configuration

The static key configuration of OpenVPN is the most basic type and only allows one client to connect to one server. You can use this simple set-up if you want to set up VPN connectivity between your laptop or home computer and one of your servers on the Internet, somewhere. This is the quickest configuration to set up. However, we need to make sure that after the key is generated at one end, it is securely transferred to the other end before initiating the connection. This is only a one-time job, so the effort is worth it.

To start with, we will generate the static key:

```
root@vpn.unixclinic.net # cd /etc/openvpn/keys
root@vpn.unixclinic.net # /usr/sbin/openvpn --genkey --secret static.key
root@vpn.unixclinic.net # ls -lZ
-rw------- root root user_u:object_r:openvpn_etc_rw_t static.key
```

Now let us configure the server part:

```
root@vpn.unixclinic.net # vi /etc/openvpn/static-server.conf
port 1194
proto udp
dev tun

# The keep alive directive is particularly important if you are using UDP
# through a stateful firewall like Netfilter. Because UDP is connectionless
# any stateful firewall will forget about the connection if packets are not
# going through it at regular intervals.
keepalive 10 60

ping-timer-rem
persist-tun
persist-key

# Enable compression (use only if compiled with lzo support)
comp-lzo

# Short log of active connections and internal routing table.
# Recreated every minute.
status openvpn-status.log

log-append  openvpn.log
```